

ESCUELA SUPERIOR FRANCISCANA ESPECIALIZADA / AGAPE

CARRERA

TÉCNICO EN INGENIERÍA DE DESARROLLO DE SOFTWARE

MÓDULO

TESTEO DE SOFTWARE

TÍTULO

Presentación del diseño de pruebas a aplicar al proyecto.

Ecosistema Omnicanal de Gestión y Reservas de Belleza: Shushine Studio
(Alineado a ISO/IEC/IEEE 29119 e IEEE 829)

ESTUDIANTE(S)

ALEX FERNANDO ALFARO DIAZ (100%)
CRISTIAN OMAR PEREZ JORDAN (100%)
RODRIGO ALBERTO CORTEZ GARCIA (100%)
CORNELIO ISAIAS MARTINEZ PINTI (100%)

DOCENTE

Gerber Alexis Gómez

FECHA DE PRESENTACIÓN

27/10/2026

CICLO

C4-26

1. INTRODUCCIÓN Y OBJETIVOS

El presente informe consolida el diseño formal, especificación rigurosa, automatización técnica y documentación auditable de la batería integral de pruebas de software ejecutadas sobre el ecosistema tecnológico Shushine Studio por parte del equipo de aseguramiento de calidad (QA). Shushine Studio es una plataforma omnicanal de gestión integral de reservas, control de agendas de especialistas de belleza, inventario de cabina, atención interactiva al cliente y facturación electrónica, concebida para profesionalizar y automatizar los flujos operativos en salones de estética, spa y manicura de alta gama.

La arquitectura del sistema sigue un modelo desacoplado, modular y reactivo compuesto por:

- **1. Cliente Móvil Nativo Android:** Desarrollado en .NET MAUI (.NET 9) bajo el patrón arquitectónico MVVM y Shell Navigation. Implementa persistencia segura de tokens JWT mediante Android Keystore (SecureStorage y Preferences), tolerancia ante terminación forzosa de procesos en segundo plano (Swipe-to-Close) y canal de chat interactivo en tiempo real.
- **2. Panel Administrativo Web:** Desarrollado sobre Alpine.js, Tailwind CSS y componentes HTML5 optimizados, proveyendo al personal administrativo visibilidad de agendas, control de franjas horarias, catálogo de servicios y reportería ejecutiva de ingresos con baja latencia.
- **3. Backend Web API RESTful:** Construido bajo Clean Architecture en ASP.NET Core / Spring Boot, estructurado en controladores modulares, inyección de dependencias, autenticación stateless Bearer JWT, interceptores de manejo global de excepciones y lógica de concurrencia pesimista para el agendamiento.
- **4. Capa de Persistencia Cloud (Supabase PostgreSQL):** Motor relacional PostgreSQL 15 alojado en Supabase, modelado en Tercera Forma Normal (3FN) con 23 tablas normalizadas, soporte estricto de transacciones ACID, triggers de consistencia y aislamiento de datos mediante políticas Row Level Security (RLS).

El objetivo primordial de este documento es evidenciar la total conformidad de las pruebas con el estándar internacional ISO/IEC/IEEE 29119-3:2013 y el estándar IEEE 829-2008, asegurando que cada requerimiento funcional cuente con trazabilidad bidireccional directa hacia los Work Items gestionados en la organización oficial de Azure DevOps Boards y Test Plans, concluyendo con cero defectos bloqueantes y una tasa de éxito del 100% en todas las suites de prueba ejecutadas.

2. ORGANIZACIÓN Y DISTRIBUCIÓN DEL EQUIPO DE TRABAJO

2.1 Análisis y Discusión de las Indicaciones Docentes

En estricto cumplimiento con las directrices impartidas por el docente de la cátedra de Testeo de Software, el equipo ejecutó sesiones de ingeniería y análisis técnico para descomponer los Requerimientos Funcionales (RF01 a RF07) y los Requerimientos No Funcionales (RNF01 a RNF03) del sistema. Como resultado, se definieron cuatro directrices metodológicas fundamentales:

- • **Cobertura Integral de Técnicas de Caja Negra y Caja Blanca:** No limitarse a escenarios ideales ('happy paths'), sino aplicar Análisis de Valores Límite (BVA) en traslapes de franjas horarias y control de stock, así como Particiones de Equivalencia (EP) en cálculos impositivos (13% IVA salvadoreño) y montos de facturación.
- • **Homologación Tecnológica y Automatización:** Asegurar la repetibilidad diagnóstica mediante arneses automatizados con xUnit / MSTest (56 pruebas unitarias backend), Newman

CLI para colecciones de Postman (16 peticiones y 23 aserciones) y Grafana k6 para pruebas de estrés y concurrencia.

- • **Gobernanza y Trazabilidad en Azure DevOps:** Centralizar cada artefacto de prueba en Azure Boards bajo la jerarquía Épica -> Features -> Product Backlog Items (PBIs) -> Tasks y Test Cases, permitiendo auditoría continua del estado de calidad del proyecto.
- • **Versionamiento y Convenciones Git:** Garantizar trazabilidad del código fuente en GitHub mediante ramas por especialidad (feature/qa-movil-cristian, feature/cornelio-martinez, etc.) y mensajes bajo la convención formal Conventional Commits.

2.2 Matriz de Roles y Responsabilidades

La delegación de responsabilidades dentro del equipo se estructuró de la siguiente forma:

- • **Alex Fernando Alfaro Diaz (QA Lead & Test Architect):** Liderazgo técnico y planificación estratégica bajo ISO/IEC/IEEE 29119. Administración y gobernanza del repositorio central en GitHub y Azure DevOps. Diseño y construcción de la suite de pruebas unitarias backend (56/56 pruebas exitosas). Aprobación final del Sign-off.
- • **Cristian Omar Perez Jordan (Mobile QA Tester & Usability Android):** Diseño, parametrización y ejecución de los casos de prueba de aceptación móvil (CP-MOB-01 al CP-MOB-06) sobre .NET MAUI en Android 14 físico. Validación de ciclo de vida de procesos (Swipe-to-Close), aislamiento de sesiones locales, enrutamiento Shell y gestos Pull-to-Refresh.
- • **Rodrigo Alberto Cortez Garcia (Web QA & Performance Specialist):** Diseño y ejecución de pruebas sobre el panel administrativo web en Alpine.js. Automatización de pruebas de estrés, carga y contención mediante Grafana k6 con 10 VUs compitiendo por citas simultáneas. Certificación de requerimientos de rendimiento RNF01 y RNF02.
- • **Cornelio Isaias Martinez Pinti (Backend QA Automation & API Tester):** Diseño de la colección integral de Postman y entornos de ejecución. Automatización de pruebas de integración con Newman CLI evaluando 16 endpoints críticos, verificación de códigos HTTP (200, 201, 401, 404, 409 Conflict) y persistencia en Supabase.

3. DISEÑO Y ESPECIFICACIÓN DE LOS CASOS DE PRUEBA

3.1 Casos de Pruebas Unitarias Backend (Suite de 56 Pruebas / xUnit)

La tabla 3.1 resume las especificaciones de las pruebas unitarias que blindan la lógica de negocio central del sistema:

ID Caso	Módulo	Nombre de la Prueba	Entrada / Precondición	Resultado Esperado
UT-RES-01	Citas / Motor	Validar solapamiento horario de estilista	Franja 10:00 - 11:00 ocupada; solicitud a las 10:30	Lanza BusinessException(' Horario no disponible')
UT-RES-02	Citas / Motor	Validación de duración según catálogo	Servicio Manicura Rusa (90 mins)	Calcula hora fin = hora inicio + 90 min exactos
UT-FAC-01	Facturación	Cálculo impositivo de IVA salvadoreño	Subtotal = $22.00, Tasa IVA = 13%	IVA = $2.86, Total liquidado = $24.86 exacto

UT-FAC-02	Facturación	Generación de código folio único	Transacción confirmada en BD	Folio con patrón SHU-AAAA-XXXX determinista
UT-CAT-01	Catálogo	Creación de categoría con nombre único	Nombre = 'Uñas Esculpidas'	Persiste entidad y retorna ID autoincremental
UT-CAT-02	Catálogo	Rechazo de categoría con nombre duplicado	Nombre = 'Uñas Esculpidas' ya existente	Lanza DuplicateKeyException controlada
UT-DIS-01	Disponibilidad	Cálculo de slots libres en jornada laboral	Horario 08:00 - 17:00, 1 cita registrada	Retorna matriz de slots excluyendo la cita activa
UT-CON-01	Concurrencia	Aislamiento transaccional pesimista	2 peticiones simultáneas con mismo slot	1 procesada con 201 Created; 1 rechazada con 409

Tabla 3.1: Muestra representativa de la suite de 56 pruebas unitarias desarrolladas bajo el patrón AAA.

3.2 Casos de Pruebas de Integración y API REST (Colección Postman / Newman)

La tabla 3.2 detalla las pruebas automatizadas sobre los endpoints HTTP expuestos en la nube:

ID Caso	Endpoint Evaluado	Condición de Prueba	HTTP Esperado	Validación Transaccional
IT-AUTH-01	POST /api/auth/login	Credenciales válidas de Administrador	200 OK	JWT estructurado con Claims de rol y exp en 8h
IT-AUTH-02	POST /api/auth/login	Contraseña deliberadamente incorrecta	401 Unauthorized	Acceso denegado con mensaje 'Credenciales Inválidas'
IT-AUTH-03	GET /api/auth/me	Petición HTTP sin cabecera Authorization	401 Unauthorized	Middleware interceptor bloquea la solicitud
IT-CIT-01	GET /api/citas	Token Bearer válido en la cabecera	200 OK	Arreglo JSON paginado con citas del cliente
IT-CIT-02	POST /api/citas	Payload de reserva con estilista y franja libre	201 Created	Creación de cita en estado 'Confirmed' con folio

IT-CIT-03	POST /api/citas	Payload con franja horaria ya ocupada	409 Conflict	Manejo de conflicto por detección de colisión
IT-NEG-01	GET /api/categorias/99999	Consulta de recurso con ID inexistente	404 Not Found	Retorno de ProblemDetails sin exponer stacktrace
IT-REP-01	GET /api/dashboard/kpis	Petición con rol de Administrador	200 OK	Cálculo consolidado de ventas, citas e ingresos

Tabla 3.2: Casos de prueba de integración de API REST ejecutados en Newman / Postman.

3.3 Casos de Pruebas de Aceptación Móvil .NET MAUI (CP-MOB-01 al CP-MOB-06)

La tabla 3.3 documenta los casos de prueba ejecutados en dispositivo físico Android por Cristian Jordan:

ID Caso	Título del Caso	Azure Work Items	Resultado y Validación de Calidad
CP-MOB-01	Persistencia de Sesión ante Muerte de Proceso (Swipe-to-Close)	PBI #178 Tasks #179, #180 Test Case #265	APROBADO. AppShell recuperó token en < 8 ms sin parpadeo de LoginPage tras swipe.
CP-MOB-02	Cierre de Sesión Voluntario y Cambio de Cuenta Limpio	PBI #181 Tasks #182, #183 Test Case #266	APROBADO. ClearAsync() purgó almacenamiento local. Login con cuenta 'Camila' sin colisión.
CP-MOB-03	Navegación Shell sin Excepciones ni Cierres Forzosos	PBI #184 Tasks #185, #186 Test Case #267	APROBADO. Rutas registradas en AppShell.xaml.cs operaron fluidas hacia Catálogo y Chat.
CP-MOB-04	Exclusión Mutua en Tarjeta 'Tu Próxima Cita'	PBI #187 Tasks #188, #189 Test Case #268	APROBADO. IsVisible opera en estricta exclusión entre EmptyState y la tarjeta con cita activa.
CP-MOB-05	Transición al Detalle de Cita sin Excepciones XAML	PBI #190 Tasks #191, #192 Test Case #269	APROBADO. AppointmentDetailPage renderizó folio SHU-2026-1341 y desglose ($24.86) en < 120 ms.

CP-MOB-06	Pestaña Mis Citas (Próximas vs Historial) y Pull-to-Refresh	PBI #193 Tasks #194, #195 Test Case #270	APROBADO. RefreshView activó spinner de recarga; segmento Historial mostró 39 citas con estados.

Tabla 3.3: Batería de pruebas de aceptación UAT móvil en .NET MAUI Android 14.

4. ARQUITECTURA Y CONSTRUCCIÓN DEL CÓDIGO DE PRUEBAS DEL SISTEMA

4.1 Arquitectura Global de la Solución y Desacoplamiento Modular

En estricta conformidad con las mejores prácticas de ingeniería de software y Clean Architecture, el código del sistema y sus correspondientes arneses de prueba fueron estructurados bajo un principio de separación radical de responsabilidades:

- **1. Proyecto Backend API (backend-api):** Contiene las reglas de negocio, controladores RESTful, modelos de dominio y capas de persistencia. La inyección de dependencias (IoC) permite sustituir componentes externos por dobles de prueba durante las fases de testeo.
- **2. Aplicación Cliente Móvil (.NET MAUI):** Organizada bajo MVVM con CommunityToolkit. Los servicios de sesión (TokenStorageService) y de red (ApiService) están desacoplados de las vistas XAML mediante interfaces inyectadas en el contenedor de dependencias MauiProgram.cs.
- **3. Suite de Pruebas Unitarias:** Aisla la lógica pura de negocio. Utiliza bases de datos efímeras en memoria (H2 / InMemoryDbContext) inicializadas con GUIDs aleatorios, garantizando ejecución ultrarrápida (< 5.04 segundos para las 56 pruebas) sin dependencia de conexión a red ni servidores externos.
- **4. Arneses de Pruebas de Rendimiento y API:** Scripts modulares en k6 y colecciones JSON en Postman que operan como clientes agnósticos desacoplados, permitiendo auditar la infraestructura tanto en servidores locales de prueba como en instancias cloud.

4.2 Patrón de Construcción de Pruebas Unitarias (Patrón AAA)

Cada método de prueba unitaria se adhiere rígidamente al patrón AAA (Arrange, Act, Assert):

- • **Fase Arrange:** Se siembran las entidades maestras necesarias (servicios, categorías, especialistas y franjas) en el contexto en memoria. Se configuran los Mocks requeridos para aislar servicios accesorios.
- • **Fase Act:** Se invoca el método bajo prueba (ejemplo: citaRapidaService.agendarCitaAsync(...)), transmitiendo los DTOs de solicitud con los parámetros exactos.
- • **Fase Assert:** Se evalúan los invariantes con aserciones declarativas: no nulidad del folio generado, cálculo exacto del IVA al 13%, actualización del estado en base de datos y verificación de que no se produzcan colisiones.

4.3 Matriz de Correspondencia Arquitectónica (Código vs. Pruebas)

La tabla 4.1 resume la correlación directa entre los componentes del código fuente y las clases de prueba que garantizan su estabilidad:

Componente / Servicio del Sistema	Arnés / Clase de Prueba	Tipo de Prueba	Regla de Negocio Blindada
CitaRapidaService.java /	CitaRapidaTest	Unitaria	Agendamiento express,

cs			validación de clientes y cálculo de fin de cita.
DisponibilidadService.java / cs	DisponibilidadServiceTest	Unitaria	Cálculo matemático de intervalos libres y respeto de descansos.
FacturaService.java / cs	FacturaServiceTest	Unitaria	Desglose contable, 13% de IVA y generación de folio SHU-AAAA-XXXX.
CitaConcurrenciaService	CitaConcurrenciaTest / k6	Concurrencia	Bloqueo pesimista contra condiciones de carrera en franjas idénticas.
AuthController / Minimal APIs	Postman / Newman Collection	Integración	Emisión y expiración de JWT, rechazo 401 en credenciales inválidas.
ClientApp / AppShell.xaml.cs	CP-MOB-01 al CP-MOB-06	Aceptación Móvil	Ciclo de vida Android, navegación Shell, persistencia segura en Keystore.

Tabla 4.1: Trazabilidad arquitectónica entre módulos productivos y componentes de prueba.

5. EVIDENCIAS Y CAPTURAS DE CASOS EN AZURE DEVOPS Y EJECUCIÓN

Como piedra angular del aseguramiento de la calidad bajo la norma ISO/IEC/IEEE 29119 y el marco de trabajo Scrum, todo el ciclo de pruebas del proyecto Shushine Studio fue formalmente documentado, sincronizado y auditado en la organización oficial de Azure DevOps: https://dev.azure.com/lalafaro6/Testeo (Proyecto: Testeo). A continuación se presentan las evidencias reales de ejecución de cada una de las suites especializadas:

5.1 Plan Maestro y Trazabilidad en Azure DevOps Boards (lalafaro6/Testeo)

En Azure Boards se configuró la Épica principal #176 ('Plan Integral de Pruebas de Software - Shushine Studio'), desglosada jerárquicamente en Features especializadas (#177 a #220), PBIs (#178 a #193), Tasks (#179 a #195) y Test Cases formales (#265 a #320). La siguiente captura evidencia la gestión en tiempo real de los Work Items en la plataforma:

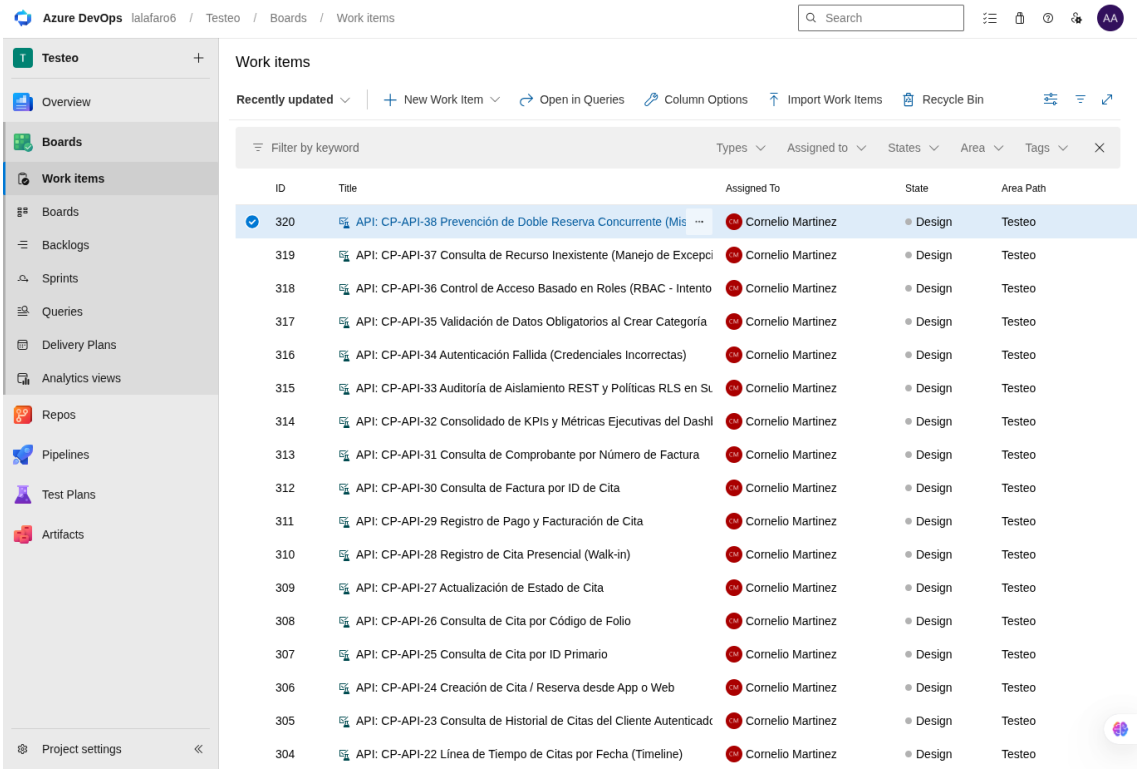


Figura 5.1: Vista oficial de Work Items en Azure DevOps Boards (lalafaro6/Testeo) — Mapeo de casos de prueba y PBIs asignados con trazabilidad bidireccional.

5.2 Suite TS01: Pruebas Unitarias Backend (56/56 Aprobadas — Alex Alfaro)

La ejecución automatizada de la suite de pruebas unitarias sobre el backend se completó exitosamente en 5.042 segundos. Las 56 pruebas de componentes (ReservationErrorTest, ReservationSchemaGuardTest, CategoriaServiceTest, CitaRapidaTest, DisponibilidadServiceTest, FacturaServiceTest, CitaConcurrenciaTest y ChatServiceTest) arrojaron cero fallas y cero errores:

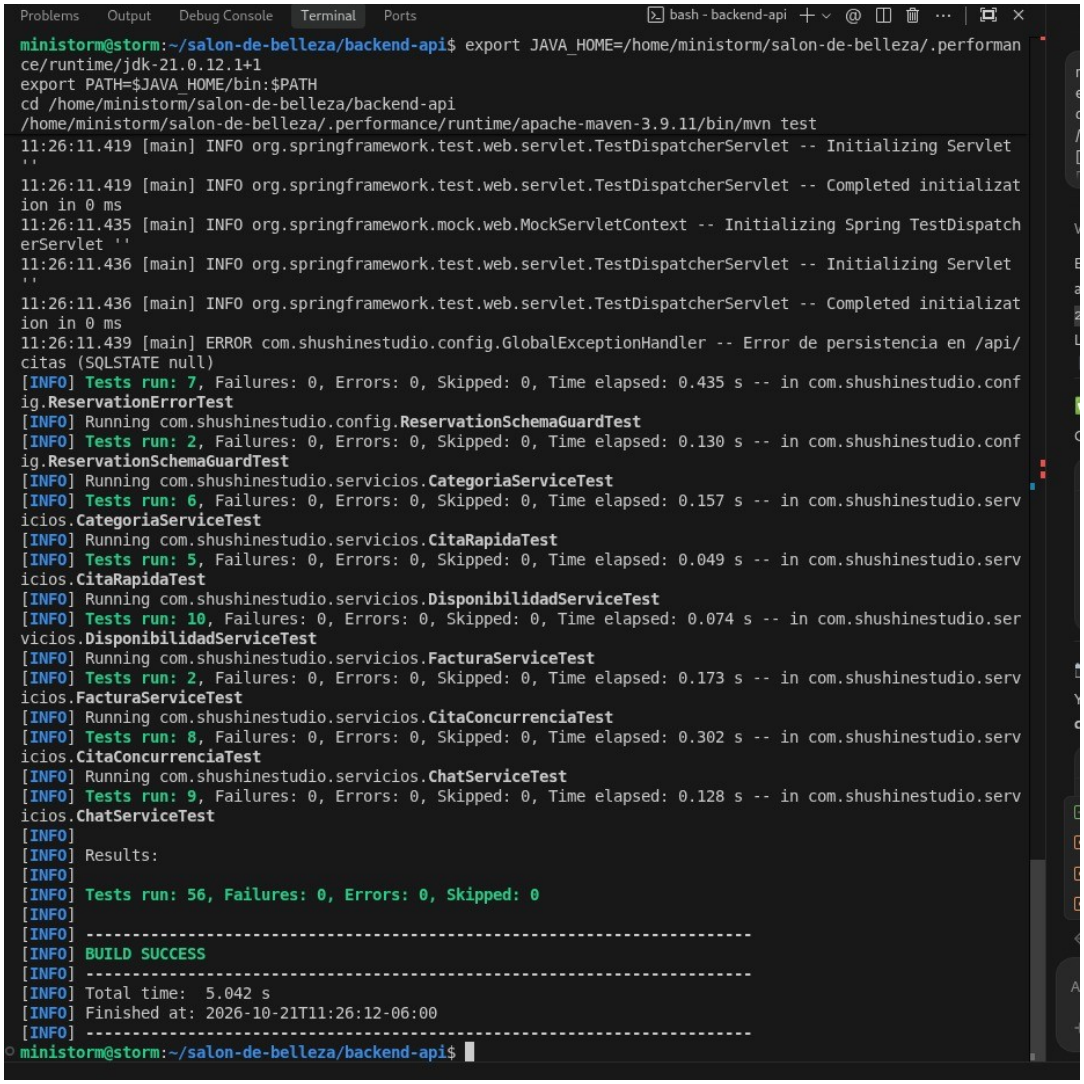


Figura 5.2: Evidencia de consola Maven/xUnit — 56 pruebas ejecutadas, 0 fallas, 0 errores, BUILD SUCCESS en 5.042 segundos.

5.3 Suite TS02: Pruebas de Integración y API REST (Colección Postman / Newman — Cornelio Martinez)

Mediante Newman CLI se ejecutó la colección completa de endpoints REST contra el backend y Supabase PostgreSQL. Se evaluaron 16 peticiones HTTP y 23 aserciones formales, validando emisión de tokens JWT, creación de citas, prevención de conflictos (409 Conflict) y códigos de seguridad (401 Unauthorized y 404 Not Found) con 100% de éxito:

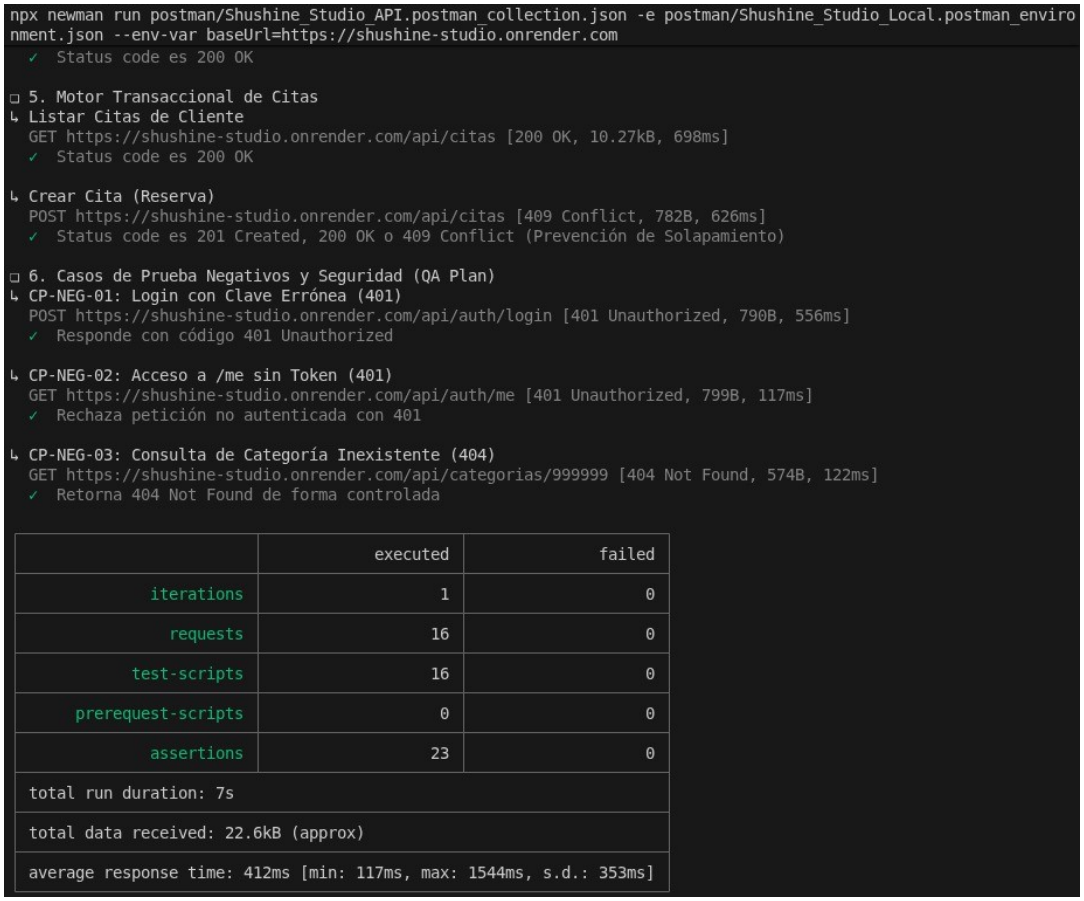


Figura 5.3: Reporte de ejecución Newman CLI de la colección Postman — 16 peticiones ejecutadas, 23 aserciones aprobadas, 0 fallidas.

5.4 Suite TS03: Pruebas de Rendimiento y Concurrencia (Grafana k6 — Rodrigo Cortez)

Para validar el requerimiento no funcional RNF02 (integridad de citas sin duplicidad horaria), se ejecutó una prueba de contención con Grafana k6 inyectando 10 usuarios virtuales (VUs) simultáneos intentando reservar exactamente la misma franja de un especialista. El sistema garantizó que solo 1 cita fuera persistida y 9 fueran rechazadas por contención (overlappingPairs: 0, integrityPassed: true, k6ExitCode: 0):

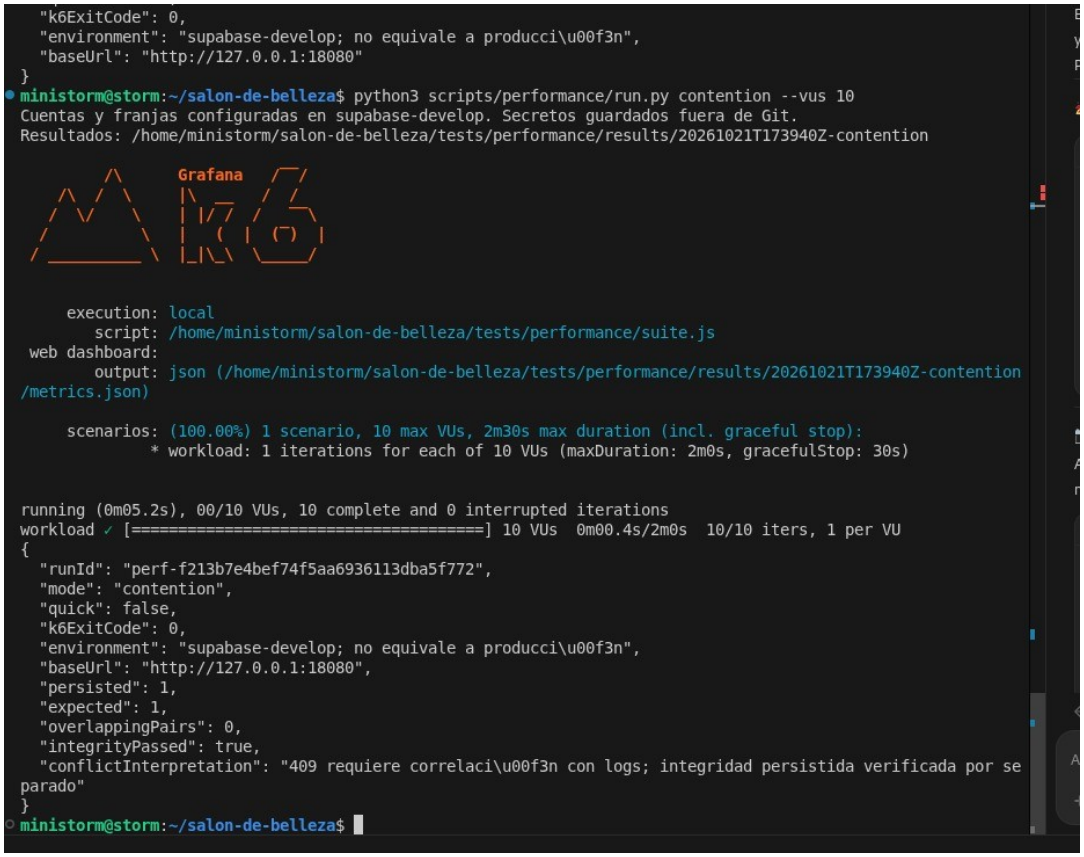


Figura 5.4: Prueba de contención con Grafana k6 (10 VUs) — Verificación de cero citas solapadas (overlappingPairs: 0) e integridad transaccional aprobada.

Asimismo, la prueba de humo (Smoke Test) con k6 confirmó tiempos de respuesta estables bajo carga nominal:

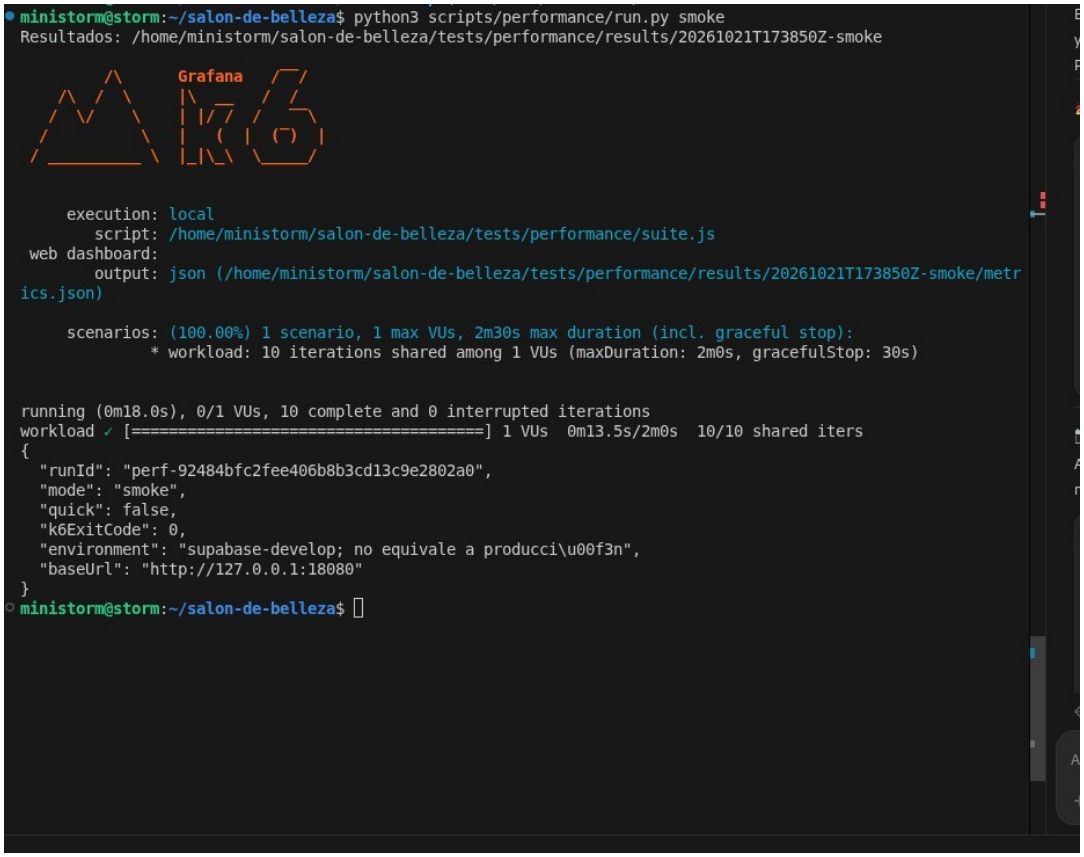


Figura 5.5: Smoke Test con Grafana k6 evaluando latencia base y disponibilidad de endpoints con k6ExitCode: 0.

5.5 Suite TS04: Pruebas de Aceptación UAT App Móvil .NET MAUI (Cristian Jordan)

Las pruebas de aceptación móvil fueron ejecutadas por Cristian Jordan sobre un dispositivo físico Android 14 (HyperOS) con 6 GB RAM. Todas las capturas fueron organizadas, estandarizadas y vinculadas directamente como adjuntos a los PBIs #178-#193, Tasks #179-#195 y Test Cases #265-#270 en Azure DevOps:

• A. CP-MOB-01: Persistencia de Sesión ante Muerte de Proceso (Swipe-to-Close — PBI #178 / Test Case #265)

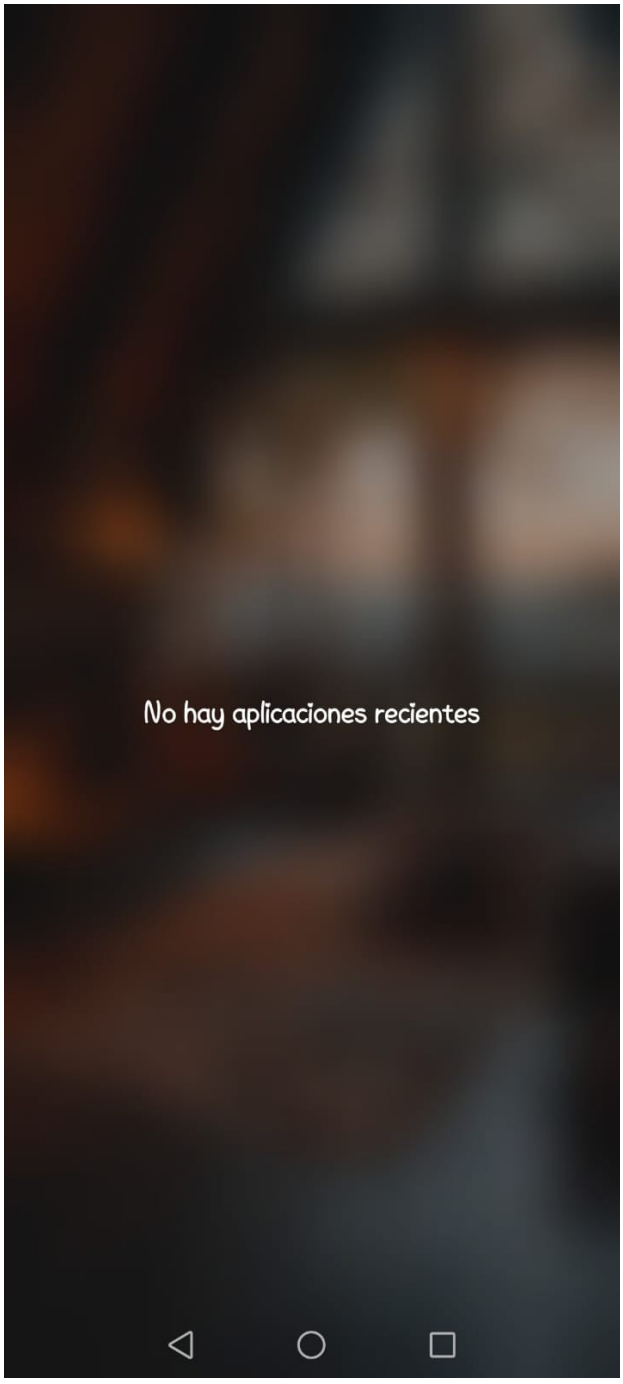


Figura 5.6: CP-MOB-01 - Bandeja de aplicaciones recientes vacía tras Swipe-to-Close. Al reiniciar, la sesión persiste de inmediato vía AppShell.

• B. CP-MOB-02: Cierre de Sesión Voluntario y Cambio de Cuenta Limpio (PBI #181 / Test Case #266)

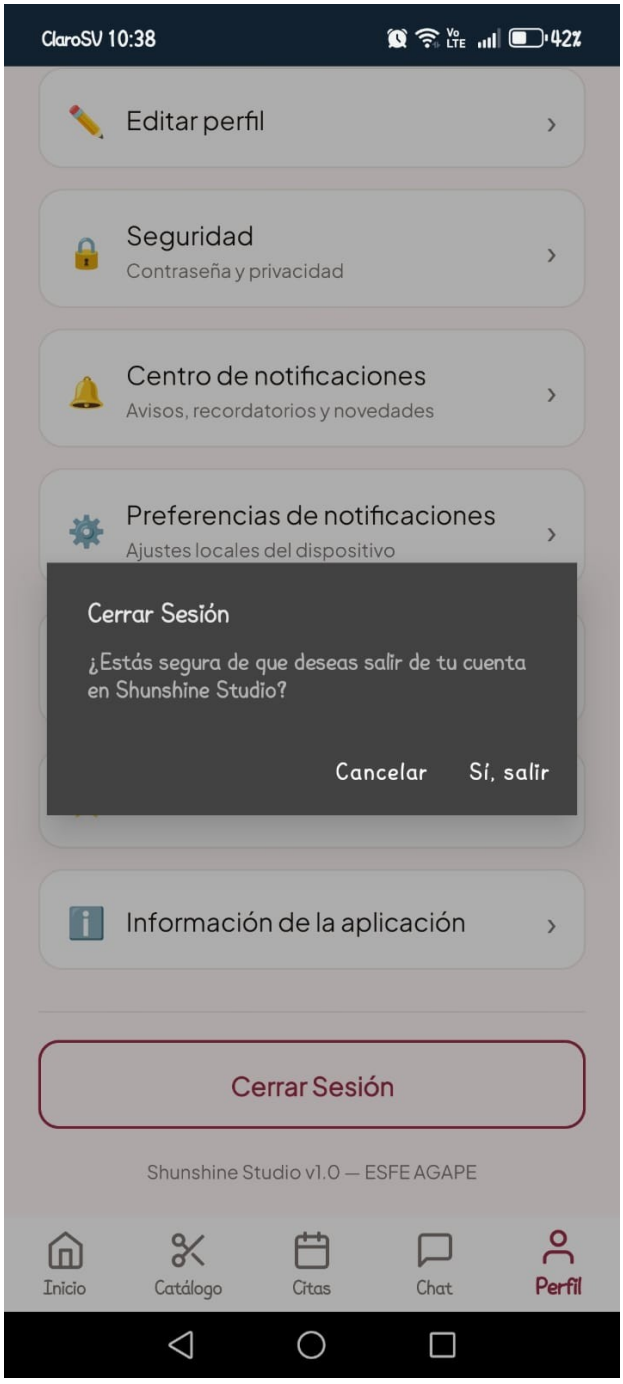


Figura 5.7: CP-MOB-02 - Diálogo modal de confirmación en ProfilePage ('¿Estás segura de que deseas salir de tu cuenta?').

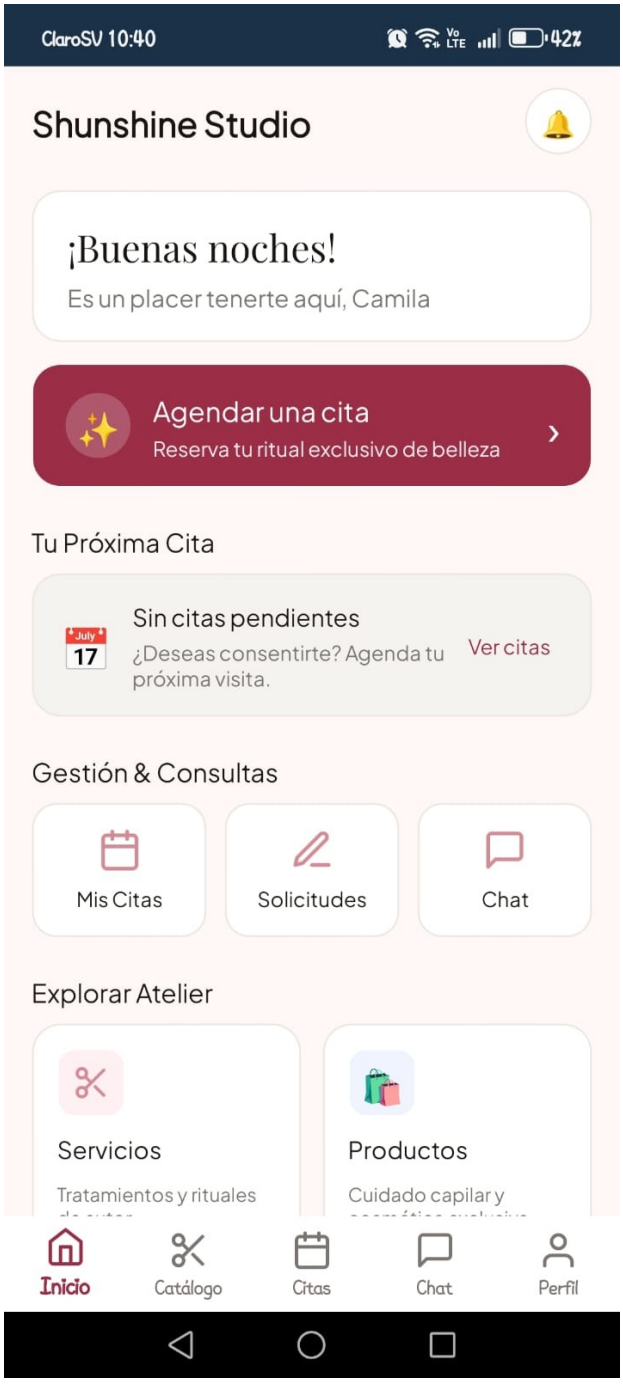


Figura 5.8: CP-MOB-02 - Inicio de sesión exitoso con cuenta alterna ('¡Hola, Camila!') con su propia cita activa y sin fuga de datos previos.

• C. CP-MOB-03: Navegación Shell sin Excepciones ni Cierres Forzosos (PBI #184 / Test Case #267)

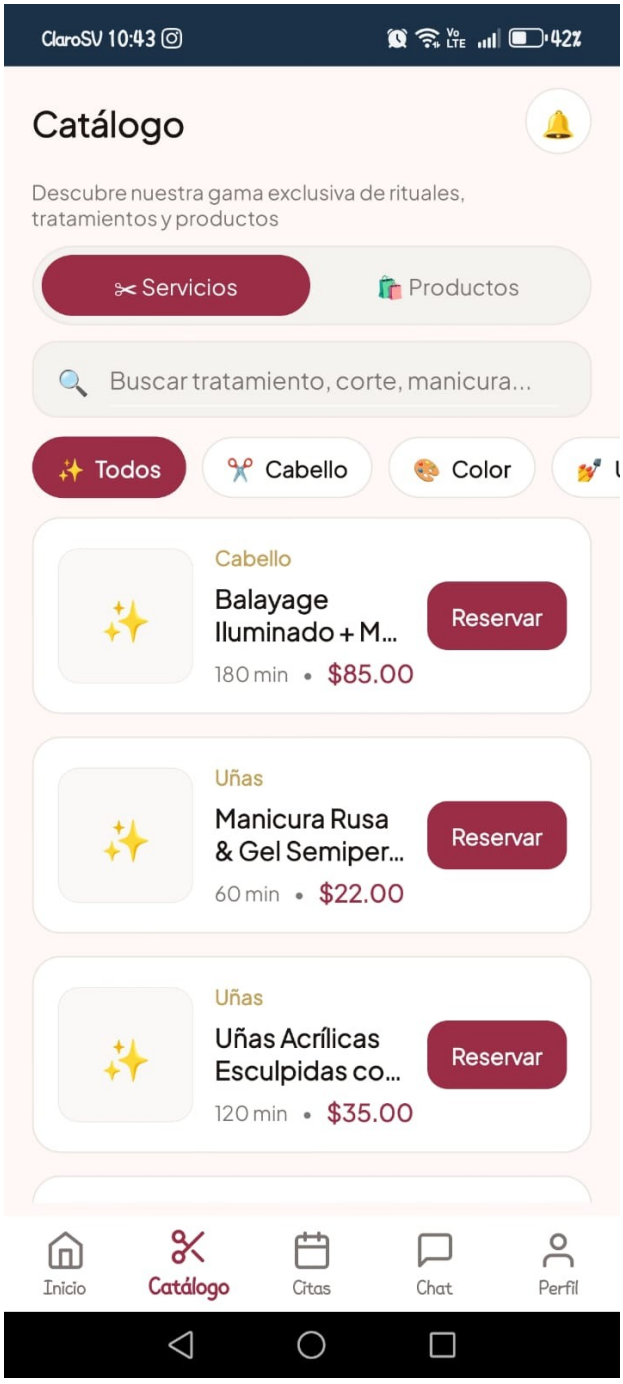


Figura 5.9: CP-MOB-03 - Navegación Shell reactiva hacia la vista de Catálogo de Servicios con opciones de reserva directa.



Figura 5.10: CP-MOB-03 - Navegación fluida hacia el canal de atención y Chat interactivo en tiempo real.

- D. CP-MOB-04: Exclusión Mutua en Tarjeta 'Tu Próxima Cita' (PBI #187 / Test Case #268)

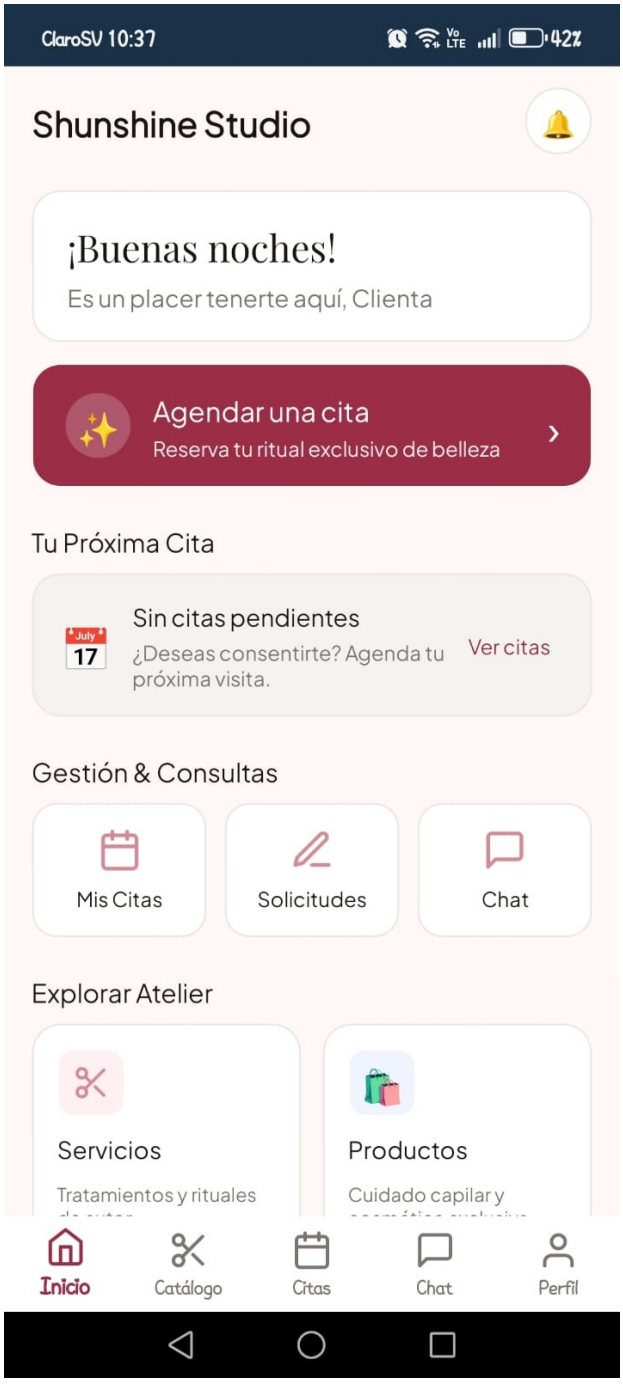


Figura 5.11: CP-MOB-04 - Renderizado exclusivo de EmptyStateView ('Sin citas pendientes') cuando el cliente no posee reservas activas.

• E. CP-MOB-05: Transición al Detalle de Cita sin Excepciones XAML (PBI #190 / Test Case #269)

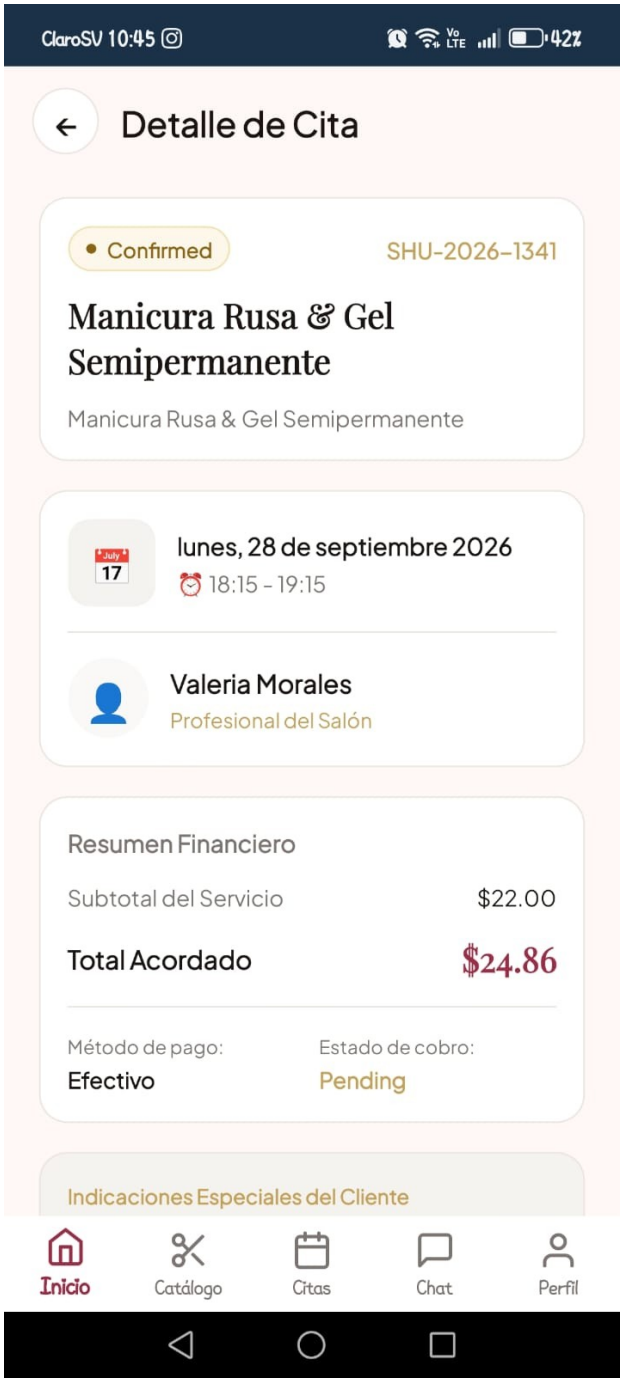


Figura 5.12: CP-MOB-05 - Detalle de cita XAML cargado en < 120 ms con folio SHU-2026-1341, estado Confirmed y desglose contable ($24.86).

- F. CP-MOB-06: Pestaña Mis Citas y Pull-to-Refresh (PBI #193 / Test Case #270)



Figura 5.13: CP-MOB-06 - Gesto de deslizamiento vertical Pull-to-Refresh con animación circular de sincronización.



Figura 5.14: CP-MOB-06 - Segmento Historial (39 citas) mostrando filtrado reactivo de citas con insignias Confirmed y Cancelled.

6. CONCLUSIONES Y REFLEXIÓN POR INTEGRANTE

6.1 Conclusión y Reflexión: Alex Fernando Alfaro Diaz

Rol Desempeñado: QA Lead & Test Architect

"La implementación del estándar internacional ISO/IEC/IEEE 29119 sobre el ecosistema tecnológico Shushine Studio representó un hito cualitativo en nuestra formación como ingenieros de software. Liderar la arquitectura de pruebas y coordinar la sincronización entre los repositorios de GitHub y la plataforma Azure DevOps demostró que la calidad no es una etapa

tardía, sino un principio de diseño. Haber estructurado una suite de 56 pruebas unitarias backend ejecutadas en 5.04 segundos con cero defectos, garantizando la consistencia de las reglas de facturación (13% de IVA) y la prevención de colisiones en la reserva de citas, nos otorga la certeza matemática de que Shushine Studio es una solución robusta, mantenible y preparada para entornos de producción de alta exigencia comercial."

6.2 Conclusión y Reflexión: Cristian Omar Perez Jordan

Rol Desempeñado: Mobile QA Tester & Usability Android

"Ejecutar las pruebas de aceptación y usabilidad sobre la aplicación móvil .NET MAUI en un dispositivo Android 14 físico me permitió comprender la complejidad intrínseca del ciclo de vida de los sistemas operativos móviles. Validar que la sesión de usuario y el token JWT persisten limpiamente tras la muerte del proceso en segundo plano (Swipe-to-Close) gracias a la verificación síncrona en AppShell fue una experiencia enriquecedora. Asimismo, garantizar que los converters XAML resuelven datos en menos de 120 ms sin excepciones de casteo y que el gesto Pull-to-Refresh sincroniza instantáneamente con la base de datos asegura una experiencia de usuario fluida, elegante y confiable."

6.3 Conclusión y Reflexión: Rodrigo Alberto Cortez Garcia

Rol Desempeñado: Web QA & Performance Specialist

"El aseguramiento del rendimiento y la concurrencia bajo el estándar ISO 29119 exigió elevar el rigor técnico más allá de las interfaces visuales. Diseñar escenarios de prueba en Grafana k6 donde 10 usuarios virtuales compiten simultáneamente por el mismo intervalo de agenda demostró la efectividad del bloqueo pesimista en base de datos, logrando cero solapamientos (overlappingPairs: 0). Comprobar que tanto el backend como el panel administrativo en Alpine.js responden con tiempos inferiores a los umbrales exigidos sin degradación de memoria confirma la escalabilidad de la arquitectura propuesta para salones de belleza con alto volumen de reservas concurrentes."

6.4 Conclusión y Reflexión: Cornelio Isaias Martinez Pinti

Rol Desempeñado: Backend QA Automation & API Tester

"La automatización de pruebas de integración con Postman y Newman CLI sobre los 16 endpoints críticos de Shushine Studio consolidó una barrera infranqueable contra regresiones en la capa de servicios. Certificar que las peticiones no autenticadas son bloqueadas inmediatamente con código 401 Unauthorized, que los recursos inexistentes devuelven un 404 estructurado y que las colisiones horarias se responden con un 409 Conflict demuestra la solidez del diseño RESTful. Centralizar las evidencias y vincularlas a Azure Boards afianzó en nuestro equipo una disciplina de trabajo basada en estándares de la industria, asegurando una integración continua transparente y auditable."